
BTS SIO 2024–2026 — Option SLAM
MCCI Business School — Session 2026
Réalisation Professionnelle n°2

RAPPORT DE PROJET

VoucherManager

Application desktop JavaFX de gestion de bons cadeaux

Étudiante : **NAZRA BINTI ABDALLAH**
Organisation cliente : **ShopGroup Ltd**
Prestataire : **MultiSys SSII**

Année académique 2025–2026

SOMMAIRE

INTRODUCTION.....	3
PREMIÈRE PARTIE — LA DÉMARCHE PROJET	4
I. Présentation du contexte	4
II. Objectifs et problématiques	4
III. Missions et plan de travail.....	5
IV. Technologies utilisées	5
DEUXIÈME PARTIE — ANALYSE DU PROJET	6
I. Analyse de l'existant	6
II. Expression des besoins	6
III. Solution retenue	7
TROISIÈME PARTIE — CONCEPTION FONCTIONNELLE	8
I. Architecture de la solution	8
II. Modèle de données	8
III. Flux métier	9
QUATRIÈME PARTIE — DÉVELOPPEMENT TECHNIQUE	10
I. Environnement de développement.....	10
II. Ce qui a été développé	10
III. Sécurité de l'application	11
CINQUIÈME PARTIE — TESTS ET MISE EN PRODUCTION	13
I. Tests réalisés.....	13
II. Déploiement et livraison	13
CONCLUSION	15

INTRODUCTION

Ce rapport présente le projet VMS (Voucher Management System), réalisé dans le cadre de la Réalisation Professionnelle n°2 du BTS SIO option SLAM à la MCCI Business School, Île Maurice, pour la session 2026.

Ce deuxième projet met en scène la même configuration que le premier : d'un côté, ShopGroup Ltd, une entreprise multi-enseignes mauricienne souhaitant informatiser la gestion de ses bons cadeaux, et de l'autre, MultiSys, la SSII prestataire chargée de concevoir et de livrer la solution applicative. En tant qu'étudiant intégré à l'équipe de développement de MultiSys, j'ai été responsable de la conception et du développement complet de l'application.

VoucherManager est une application desktop développée en Java avec JavaFX. Elle prend en charge l'intégralité du cycle de vie des bons cadeaux : depuis la saisie d'une demande client jusqu'à la rédemption du bon en magasin, en passant par la validation du paiement, l'approbation, la génération automatique des bons en PDF avec QR codes, et l'envoi de notifications email aux différentes parties prenantes.

Ce rapport retrace l'ensemble de la démarche suivie : compréhension du contexte et des besoins du client, choix de la solution technique dans le cadre imposé par la formation, conception de l'architecture et de la base de données, développement des fonctionnalités, et tests de validation avant livraison.

PREMIÈRE PARTIE — LA DÉMARCHE PROJET

I. Présentation du contexte

ShopGroup Ltd est une entreprise mauricienne qui regroupe plusieurs enseignes commerciales. Pour fidéliser sa clientèle et dynamiser ses ventes, elle propose des bons cadeaux à ses clients. Ces bons, émis sur demande, permettent à leurs bénéficiaires d'effectuer des achats dans l'une ou plusieurs des enseignes du groupe.

Jusqu'à présent, la gestion de ces bons cadeaux était entièrement manuelle. Les demandes étaient collectées par email ou par téléphone, les bons imprimés et envoyés physiquement, et leur suivi tenu sur des tableurs Excel. Cette organisation engendrait des risques importants : perte de traçabilité, risques de fraude à la rédemption, absence de circuit de validation formalisé, et communication non structurée entre les différents intervenants du processus (comptable, approbateur, superviseur en magasin).

ShopGroup Ltd a donc décidé de confier à MultiSys la conception d'une application qui centralise et automatise ce processus de bout en bout. L'application doit gérer les demandes, les valider selon un circuit d'approbation structuré, générer les bons sous forme de documents PDF sécurisés avec QR codes, envoyer des notifications automatiques par email, et permettre la rédemption sécurisée en magasin.

II. Objectifs et problématiques

L'objectif principal du projet est de remplacer la gestion manuelle et dispersée des bons cadeaux par un système centralisé, traçable et sécurisé. L'application doit permettre à chaque acteur du processus d'effectuer ses tâches depuis une interface adaptée à son rôle, sans avoir besoin d'accéder aux parties du système qui ne le concernent pas.

Le processus à informatiser suit un flux séquentiel précis. Un administrateur ou un comptable crée une demande de bon cadeau pour un client. Le comptable valide ensuite le paiement, ce qui déclenche automatiquement une notification email à l'approbateur et à l'administrateur. L'approbateur examine la demande et l'approuve, ce qui lance la génération automatique des bons en PDF avec QR codes et leur envoi par email au client. Enfin, lorsque le client se présente en magasin, le superviseur scanne ou saisit le code du bon pour effectuer la rédemption, avec toutes les vérifications anti-fraude nécessaires.

Les problématiques techniques soulevées par ce projet sont multiples. Comment concevoir une architecture applicative desktop en Java qui soit maintenable et évolutive ? Comment générer des documents PDF de qualité avec des QR codes intégrés de manière programmatique ? Comment structurer un circuit d'approbation avec des notifications automatiques par email ? Et comment garantir la traçabilité complète de chaque bon depuis sa création jusqu'à sa rédemption, en empêchant toute utilisation frauduleuse ?

III. Missions et plan de travail

Ma mission sur ce projet a couvert l'intégralité du cycle de développement, de l'analyse des besoins jusqu'à la livraison de l'application. J'ai travaillé seul sur l'ensemble des composants : conception de la base de données PostgreSQL, développement de l'interface JavaFX, implémentation des services métier, intégration des bibliothèques tierces pour la génération PDF et l'envoi d'emails, et tests de validation.

Le travail a été organisé en plusieurs phases. J'ai commencé par l'analyse du cahier des charges et la modélisation de la base de données, en identifiant les entités métier et les relations entre elles. Ensuite j'ai mis en place l'environnement de développement et la structure du projet Maven avec les dépendances nécessaires. La troisième phase a porté sur le développement de la couche d'accès aux données avec les DAO, suivi du développement des interfaces JavaFX écran par écran. J'ai ensuite intégré les services techniques génération PDF avec iText7, QR codes avec ZXing, envoi d'emails avec Jakarta Mail avant de mettre en place la sécurité et les contrôles d'accès par rôle. Enfin j'ai conduit les tests fonctionnels et finalisé la documentation.

Cette organisation progressive m'a permis de construire l'application couche par couche, en m'assurant que chaque composant fonctionnait correctement avant d'aborder le suivant.

IV. Technologies utilisées

L'application VoucherManager est développée en Java 17 avec JavaFX 17.0.6 pour l'interface graphique. Java a été choisi pour sa robustesse, sa portabilité et son adéquation avec le développement d'applications desktop professionnelles. JavaFX est le framework graphique moderne pour Java, permettant de créer des interfaces riches avec une séparation claire entre la logique et la présentation.

La gestion des dépendances est assurée par Maven 3.9+, ce qui facilite l'intégration des nombreuses bibliothèques tierces utilisées dans le projet. La base de données est PostgreSQL 14+, accessible depuis Java via JDBC. L'IDE utilisé est IntelliJ IDEA, particulièrement adapté au développement Java grâce à son support natif de Maven, JavaFX et PostgreSQL.

Plusieurs bibliothèques spécialisées ont été intégrées pour les fonctionnalités avancées du projet. iText7 7.2.5 est utilisé pour la génération programmatique des documents PDF contenant les bons cadeaux. ZXing 3.5.3 prend en charge la génération des QR codes intégrés dans ces PDF. Jakarta Mail 2.0.1 gère l'envoi automatique d'emails via SMTP Gmail. jBCrypt 0.4 assure le hachage sécurisé des mots de passe. Apache POI 5.2.5 permet l'export des rapports au format Excel.

DEUXIÈME PARTIE — ANALYSE DU PROJET

I. Analyse de l'existant

L'analyse de la situation initiale de ShopGroup Ltd a mis en évidence plusieurs dysfonctionnements liés à l'absence d'un outil informatique dédié à la gestion des bons cadeaux. La création d'un bon nécessitait des échanges de mails entre le service commercial, la comptabilité et la direction, sans aucune centralisation de l'information. Il n'existait pas de circuit de validation formalisé : l'approbation se faisait de manière informelle, souvent par simple échange verbal ou par email sans archivage structuré.

La traçabilité des bons après émission était quasi inexistante. Lorsqu'un client se présentait en magasin pour utiliser son bon, le superviseur devait appeler le siège pour vérifier sa validité, ce qui rallongeait le processus et créait des risques d'erreur ou de fraude. Les bons perdus ou volés ne pouvaient pas être désactivés rapidement. L'absence de reporting consolidé empêchait la direction d'avoir une vision claire du volume de bons émis, en attente ou rédemptés.

Cette situation justifiait pleinement la mise en place d'une application centralisée avec un circuit de validation structuré, une génération sécurisée des bons, et un système de rédemption avec vérifications automatiques.

II. Expression des besoins

Le cahier des charges fourni par ShopGroup Ltd via MultiSys exprimait des besoins fonctionnels organisés autour de quatre profils utilisateurs distincts, chacun ayant ses propres responsabilités dans le processus.

L'administrateur a un accès complet à l'application. Il peut créer des demandes, gérer les comptes utilisateurs et les enseignes, consulter l'audit trail complet de toutes les opérations, et accéder aux rapports Excel. Le comptable est responsable de la validation des paiements : il confirme qu'un client a bien réglé le montant correspondant à son bon, ce qui débloque la suite du processus. L'approbateur reçoit une notification email après chaque validation de paiement et doit examiner la demande avant de l'approuver. C'est son approbation qui déclenche la génération automatique des bons PDF et leur envoi au client. Le superviseur de magasin intervient en fin de processus : il scanne ou saisit le code d'un bon lorsqu'un client souhaite l'utiliser, et l'application lui indique si le bon est valide, son montant, et l'enseigne concernée.

Sur le plan technique, le cahier des charges précisait que l'application devait être une application desktop Java, avec une base de données relationnelle, un système d'authentification sécurisé, la génération de documents PDF avec QR codes, et l'envoi automatique d'emails. L'utilisation d'au moins une procédure stockée et d'un trigger en base de données était également requise.

III. Solution retenue

Dans le cadre du BTS SIO option SLAM, le stack technique à utiliser pour ce projet était définie par les exigences de la formation : une application desktop développée en Java avec JavaFX pour l'interface graphique, et PostgreSQL comme système de gestion de base de données. Ce choix n'a donc pas résulté d'une comparaison entre plusieurs options, mais d'un cadre pédagogique établi que je me suis approprié et mis en œuvre.

Java est un langage orienté objet mature, fortement typé et particulièrement adapté au développement d'applications d'entreprise. Son écosystème riche en bibliothèques tierces — notamment iText7, ZXing et Jakarta Mail — le rend très pertinent pour un projet comme VMS qui nécessite des fonctionnalités avancées de génération documentaire et de communication. JavaFX permet de créer des interfaces graphiques desktop modernes en Java, avec une architecture qui sépare proprement la logique applicative des vues grâce au pattern MVC.

PostgreSQL est un SGBD relationnel open source reconnue pour sa robustesse, sa conformité aux standards SQL, et ses fonctionnalités avancées comme les procédures stockées, les triggers et les vues matérialisées. Il est particulièrement bien adapté à un projet nécessitant une intégrité des données forte et une traçabilité complète des opérations. La gestion des dépendances avec Maven facilite l'intégration et la mise à jour des bibliothèques du projet, ce qui est essentiel compte tenu du nombre de dépendances externes impliquées.

TROISIÈME PARTIE — CONCEPTION FONCTIONNELLE

I. Architecture de la solution

L'application VoucherManager est structurée selon le patron MVC (Modèle-Vue-Contrôleur), adapté à JavaFX. Cette architecture sépare clairement les responsabilités entre les trois couches de l'application : la couche modèle qui représente les données et la logique métier, la couche vue qui gère l'affichage via les fichiers FXML JavaFX, et la couche contrôleur qui fait le lien entre les deux en répondant aux actions de l'utilisateur.

Le code source est organisé en plusieurs packages distincts. Le package config contient la classe DatabaseConfig qui gère la connexion JDBC à PostgreSQL, ainsi que SessionManager qui maintient l'état de la session de l'utilisateur connecté — son identifiant, son prénom et son rôle — tout au long de l'exécution de l'application. Le package controller contient un contrôleur JavaFX par écran de l'application. Le package dao regroupe les classes d'accès aux données, une par entité métier, qui encapsulent toutes les requêtes SQL. Le package model contient les classes Java représentant les entités métier. Enfin, les packages service et util contiennent respectivement les services techniques (EmailService, PdfBonService) et les classes utilitaires partagées.

Cette organisation en package clair facilite la maintenance et la compréhension du code. Chaque classe a une responsabilité unique et bien définie, ce qui limite les dépendances croisées et rend le code plus facile à tester et à faire évoluer.

II. Modèle de données

La base de données vms_db est conçue pour stocker l'ensemble des informations nécessaires au bon fonctionnement du circuit de validation. Elle comprend plusieurs tables principales reliées par des contraintes de clés étrangères.

La table utilisateurs stocke les comptes des quatre profils de l'application (admin, comptable, approbateur, superviseur), avec le mot de passe haché en BCrypt, le rôle sous forme d'énumération, et les horodatages de création. La table enseignes référence les différentes boutiques du groupe pour lesquelles des bons peuvent être émis. La table clients stocke les informations des bénéficiaires des bons. La table demandes est la table centrale du système : elle enregistre chaque demande de bon cadeau avec ses montants, son statut (en attente, payé, approuvé, émis, rédemptué, annulé), et les horodatages de chaque changement d'état. La table « bons » stocke les bons individuels générés à partir d'une demande, chacun avec son code unique, son QR code associé, et son statut propre. La table audit_trail enregistre chaque opération significative effectuée dans l'application, avec l'identifiant de l'utilisateur, l'action réalisée, et l'horodatage.

En complément de ces tables, la base intègre une procédure stockée qui centralise la logique de création des bons lors de l'approbation d'une demande, en s'assurant de la cohérence des données et en enregistrant l'opération dans l'audit trail. Un trigger sur la table des connexions enregistre

automatiquement chaque authentification réussie ou échouée, contribuant à la traçabilité globale du système.

III. Flux métier

Le flux de traitement d'un bon cadeau suit un chemin séquentiel précis, chaque étape étant déclenchée par l'action d'un acteur spécifique et pouvant déclencher des notifications automatiques.

Tout commence lorsqu'un administrateur ou un comptable crée une nouvelle demande en saisissant les informations du client, le montant souhaité, l'enseigne concernée et la quantité de bons. La référence de la demande est générée automatiquement selon un format standardisé, par exemple VR0001-10 pour une demande de dix bons de la première série. La demande passe alors au statut "en attente de paiement".

Le comptable, après confirmation du règlement par le client, valide le paiement dans l'application. Cette action fait passer la demande au statut "payé" et déclenche immédiatement l'envoi automatique d'un email à l'approbateur pour l'informer qu'une demande attend son examen, ainsi qu'une copie à l'administrateur. L'approbateur se connecte, examine la demande, et peut l'approuver ou la rejeter. S'il l'approuve, la procédure stockée PostgreSQL est appelée : elle génère les bons dans la table « bons » avec leurs codes uniques, puis le service PdfBonService prend le relais pour créer le document PDF contenant tous les bons avec leurs QR codes. Ce PDF est ensuite envoyé automatiquement par email au client.

Lorsque le client se présente dans une enseigne du groupe, le superviseur ouvre l'écran de rédemption, saisit ou scanne le code du bon. L'application interroge la base de données, vérifie que le bon existe, qu'il appartient bien à l'enseigne du superviseur, qu'il est au statut "émis" et qu'il n'a pas déjà été utilisé. Si toutes ces vérifications sont passées, le bon est marqué comme rédemptué et l'opération est enregistrée dans l'audit trail.

QUATRIÈME PARTIE — DÉVELOPPEMENT TECHNIQUE

I. Environnement de développement

L'environnement de développement a été mis en place avec IntelliJ IDEA comme IDE principal. IntelliJ IDEA offre un support natif et très complet pour Java, Maven et JavaFX, avec notamment la détection automatique des dépendances Maven, l'autocomplétion avancée, le débogage intégré, et des outils de refactoring puissants. C'est l'IDE le plus adapté pour un projet Java de cette envergure.

La base de données PostgreSQL a été installée localement et administrée via pgAdmin, l'interface graphique officielle de PostgreSQL. pgAdmin a permis de concevoir les tables, d'exécuter les scripts SQL de création du schéma, de tester les requêtes et les procédures stockées, et de vérifier les données au cours du développement.

Maven a été utilisé pour la gestion des dépendances du projet. Le fichier pom.xml centralise la déclaration de toutes les bibliothèques utilisées — JavaFX, les drivers JDBC PostgreSQL, iText7, ZXing, Jakarta Mail, jBCrypt et Apache POI — ainsi que la configuration du plugin javafx-maven-plugin qui permet de lancer l'application directement depuis Maven avec la commande `mvn clean javafx:run`.

II. Ce qui a été développé

Authentification et gestion des sessions

L'écran de connexion est le point d'entrée unique de l'application. L'utilisateur saisit son email et son mot de passe. La classe `LoginController` récupère ces informations, appelle le DAO utilisateurs qui interroge la base de données, et utilise `jBCrypt` pour vérifier le mot de passe soumis par rapport au hash stocké. En cas de succès, les informations de l'utilisateur sont stockées dans le `SessionManager`, et l'application charge dynamiquement l'écran de tableau de bord adapté au rôle de l'utilisateur. Chaque écran sensible commence par une vérification du rôle via `SessionManager`, redirigeant immédiatement vers l'écran de connexion si aucune session valide n'est trouvée.

Tableau de bord adaptatif

L'un des aspects les plus importants de l'application est son adaptation à chaque profil. Plutôt que d'afficher un menu global avec des parties désactivées selon le rôle, l'application charge un écran de tableau de bord différent pour chaque profil. L'administrateur voit un dashboard avec les statistiques globales et l'accès à tous les modules. Le comptable voit uniquement les demandes en attente de validation de paiement. L'approbateur voit les demandes payées en attente d'approbation. Le superviseur voit l'écran de rédemption. Cette approche garantit que chaque utilisateur ne voit que ce qui le concerne, limitant les risques d'erreur et simplifiant l'interface.

Génération des bons PDF avec QR codes

La génération des bons cadeaux en PDF est assurée par la classe PdfBonService qui utilise la bibliothèque iText7. Pour chaque bon approuvé, le service crée une page PDF avec les informations du bon (référence, montant, enseigne, date d'expiration, conditions d'utilisation) et génère un QR code via la bibliothèque ZXing. Le QR code encode le code unique du bon, permettant au superviseur de le scanner en magasin. Tous les bons d'une même demande sont regroupés dans un seul document PDF, qui est ensuite converti en tableau d'octets pour être joint à l'email envoyé au client.

Notifications email automatiques

Le service EmailService utilise Jakarta Mail configuré avec un compte Gmail via SMTP. Pour des raisons de sécurité, l'authentification ne se fait pas avec le mot de passe du compte Gmail mais avec un mot de passe d'application généré depuis les paramètres de sécurité Google. Trois types d'emails sont envoyés automatiquement : une notification à l'approbateur et à l'administrateur lors de la validation du paiement, un email au client avec ses bons PDF en pièce jointe lors de l'approbation, et optionnellement une confirmation de rédemption. Tous ces envois sont déclenchés par les actions des utilisateurs dans l'application, sans intervention manuelle.

Rédemption sécurisée et audit trail

L'écran de rédemption du superviseur propose deux modes de saisie du code du bon : la saisie manuelle dans un champ texte, ou le scan via un lecteur de code-barres USB (qui fonctionne exactement comme une saisie clavier). Après validation, l'application effectue une série de vérifications : existence du bon dans la base, correspondance avec l'enseigne du superviseur, statut "émis" requis, et absence de rédemption antérieure. Si toutes les vérifications sont passées, le bon est marqué comme rédemptué et toutes les informations de l'opération sont enregistrées dans l'audit trail avec l'identifiant du superviseur et l'horodatage précis.

Rapports Excel

L'administrateur peut générer des rapports au format Excel via Apache POI. Ces rapports listent les demandes sur une période choisie, avec leur statut, les montants, et les dates de chaque étape du processus. Cette fonctionnalité répond à un besoin concret de ShopGroup Ltd pour le suivi financier et l'analyse de l'activité des bons cadeaux.

III. Sécurité de l'application

La sécurité de VoucherManager a été pensée à plusieurs niveaux, depuis la protection des données en base jusqu'au contrôle des accès dans l'interface.

Les mots de passe ne sont jamais stockés en clair dans la base de données. jBCrypt génère automatiquement un sel unique pour chaque mot de passe et produit un hash irréversible. Cette approche garantit que même en cas de compromission de la base de données, les mots de passe des utilisateurs restent protégés.

Toutes les interactions avec la base de données se font via des requêtes JDBC préparées avec des paramètres liés, ce qui neutralise les tentatives d'injection SQL. Les données reçues depuis les formulaires JavaFX sont systématiquement validées avant traitement, tant sur la forme (champs obligatoires, formats) que sur le fond (vérification de l'existence des entités référencées).

Le contrôle d'accès par rôle est implémenté via le SessionManager. Chaque contrôleur vérifie en début d'exécution que l'utilisateur connecté dispose des droits nécessaires pour effectuer l'opération demandée. Un comptable ne peut pas approuver une demande, et un superviseur ne peut pas valider un paiement. Cette vérification est effectuée côté application, en complément des contraintes définies en base de données.

Enfin, l'audit trail enregistre chaque opération significative avec l'identifiant de l'acteur, la nature de l'action et l'horodatage. Ce journal est consultable par l'administrateur depuis l'interface et constitue la mémoire complète de toutes les opérations effectuées sur les bons cadeaux.

CINQUIÈME PARTIE — TESTS ET MISE EN PRODUCTION

I. Tests réalisés

Les tests ont été conduits de manière progressive tout au long du développement, en validant chaque fonctionnalité dès sa réalisation plutôt qu'en réservant les tests à la fin du projet.

Pour l'authentification, j'ai testé la connexion avec des identifiants valides pour chacun des quatre profils, avec un email inexistant, avec un mot de passe incorrect, et la cohérence de l'affichage du tableau de bord selon le rôle connecté. Le contrôle d'accès a été vérifié en tentant d'appeler directement les méthodes de service réservées à d'autres rôles, confirmant que les vérifications SessionManager bloquent bien les accès non autorisés.

Le flux complet du bon cadeau a été testé de bout en bout plusieurs fois, en suivant le chemin nominal : création de demande, validation du paiement avec vérification de l'envoi email, approbation avec vérification de la génération PDF et de l'envoi au client, puis rédemption avec vérification de la mise à jour du statut et de l'enregistrement dans l'audit trail. J'ai également testé les cas d'erreur : tentative de rédemption d'un bon déjà utilisé, tentative de rédemption dans une enseigne différente de celle assignée au bon, et tentative d'approbation d'une demande non payée.

La génération PDF a fait l'objet de tests spécifiques pour vérifier la lisibilité des QR codes générés, en testant leur scan avec différents lecteurs. Les emails ont été testés en envoyant des notifications réelles vers des adresses de test, en vérifiant la présence des pièces jointes et la lisibilité du contenu.

Les tests de performance ont porté sur la pagination des listes de demandes, en s'assurant que l'affichage restait fluide avec un volume de données représentatif. Les requêtes SQL ont été analysées avec pgAdmin pour identifier et corriger les éventuels problèmes de performance.

II. Déploiement et livraison

L'application VoucherManager est livrée sous forme d'un projet Maven complet, hébergé sur un dépôt Git. La procédure de déploiement se déroule en quatre étapes. Il faut d'abord s'assurer que Java 17+, Maven 3.9+ et PostgreSQL 14+ sont installés sur la machine cible. Ensuite, le script SQL fourni dans le répertoire `src/main/resources/sql/` doit être exécuté dans pgAdmin pour créer le schéma de la base de données `vms_db`. La configuration de la connexion à la base de données dans `DatabaseConfig.java` doit ensuite être ajustée avec les paramètres de l'environnement cible. Enfin, l'application se lance avec la commande `mvn clean javafx:run` depuis la racine du projet.

Pour la configuration de l'envoi d'email, `EmailService.java` doit être renseigné avec une adresse Gmail et un mot de passe d'application Google App Password, généré depuis les paramètres de sécurité du compte Google. Ce mot de passe d'application est différent du mot de passe habituel du compte et est spécifiquement conçu pour être utilisé dans des applications tierces.

Un jeu de données de démonstration est inclus dans le script SQL, avec quatre comptes utilisateurs préconfigurés — un par rôle — et quelques demandes à différents stades du processus, permettant de tester immédiatement l'ensemble des fonctionnalités de l'application après installation. Les identifiants des comptes de démonstration sont documentés dans le README du projet.

CONCLUSION

La réalisation du projet VoucherManager a été une expérience de développement plus complexe et plus exigeante que le projet SuperCar. Le passage du développement web à une application desktop Java m'a demandé de maîtriser un nouvel écosystème technologique, avec ses propres paradigmes et ses propres contraintes.

L'intégration de bibliothèques tierces comme iText7, ZXing et Jakarta Mail a été l'un des aspects les plus enrichissants du projet. Ces bibliothèques sont utilisées dans des contextes professionnels réels, et apprendre à les intégrer dans un projet Maven, à comprendre leurs APIs et à les combiner pour produire des bons cadeaux PDF avec QR codes envoyés automatiquement par email, m'a donné une vision concrète de ce que signifie développer une application d'entreprise complète.

La modélisation du circuit d'approbation a également été un exercice formateur. Réfléchir à la façon dont les différents acteurs interagissent avec le système, définir les états possibles d'une demande et les transitions entre ces états, et s'assurer que chaque transition s'accompagne des notifications et des actions appropriées, sont des préoccupations que l'on retrouve dans la grande majorité des applications de gestion d'entreprise.

Sur le plan de la sécurité, ce projet m'a permis d'approfondir ma compréhension des bonnes pratiques applicables en dehors du contexte web : hachage des mots de passe avec BCrypt, contrôle d'accès par rôle à chaque niveau de l'application, requêtes préparées pour prévenir les injections SQL, et traçabilité complète via l'audit trail.

À l'issue de ces deux projets professionnels, je mesure le chemin parcouru depuis le début de la formation. Le projet VoucherManager, par sa complexité et la diversité de ses composants techniques, me semble particulièrement représentatif des compétences attendues d'un développeur BTS SIO SLAM : capacité à analyser un besoin métier, à concevoir une architecture appropriée, à intégrer des technologies diverses, et à livrer une application sécurisée et documentée.